Scalability & Future Plan

Date	05 July 2026
Team ID	SWTID-2026-5245
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	The crop recommendation model is trained once on a static historical dataset and is not automatically retrained with new agronomic data.	Recommendations may gradually lose accuracy as regional soil, climate, and crop patterns change over time.	High
2	The system currently supports single-crop classification only, with no multi-crop ranking, crop rotation, or intercropping suggestions.	Limits the depth of decision support offered to farmers and researchers who need comparative options.	Medium
3	No persistent database is used; the Flask app runs as a stateless serverless function that loads the pickled model on every cold start.	Farmer inputs and prediction history are not stored, preventing usage analytics, trend tracking, or personalization.	High

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Deployed as a single Flask app running on a Vercel serverless function; suitable for low to moderate concurrent requests.	Adopt auto-scaling serverless functions with a load balancer and add a caching layer (e.g., Redis) for frequently repeated predictions.
2	Data Storage	No database; the model files (crop_model.pkl, label_encoder.pkl) are loaded from local storage at cold start, and no user data is persisted.	Integrate a cloud database (PostgreSQL/MongoDB) to store farmer inputs and prediction history, enabling analytics for researchers and policymakers.
3	Performance	Model is reloaded from disk on every cold start; single lightweight classifier trained on a limited dataset.	Use model warm-starts/containerization (Docker), lighter inference formats (e.g., ONNX), and periodic retraining on larger, updated datasets.
4	Security	Basic Flask app with simple type-casting on form inputs; no authentication, rate limiting, or input validation.	Add server-side input validation, request rate limiting, HTTPS enforcement, and API-key or role-based access for researcher/policymaker dashboards.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Multi-crop ranking and fertilizer dosage recommendation based on NPK gap analysis.	Q4 2026	Provides farmers with broader, prioritized crop choices and precise nutrient guidance.
Phase 3	Mobile-friendly interface with regional language support and offline data entry.	Q1 2027	Improves accessibility for farmers in rural areas with limited connectivity or English proficiency.
Phase 4	Integration with real-time weather APIs and satellite/remote-sensing soil data feeds.	Q3 2027	Enables dynamic, up-to-date recommendations that adapt to real-time environmental conditions, improving accuracy and farmer trust.

Phase 2			
Phase 3			
Phase 4			